



MediGuide

Chatbot médical RAG

Assistant santé virtuel basé sur les sources officielles HAS et INCa. Pipeline RAG, comparaison d'encodeurs, orchestration multi-agents Hermes et déploiement de production.

ÉQUIPE

Yousra · Imane · Yannis · Grace

PRODUCTION

sae.amanawebagency.com

ENCADREMENT · ANNÉE

Mr FAYE & Mme AZZAG · 2025–2026

DÉPÔT

[github.com/YousraWsites](https://github.com/YousraWsites/-chatbot-medical-s6)
/-chatbot-medical-s6

Table des matières

- Contexte et objectifs
 - La SAE BUT3 S6
 - Notre projet : MediGuide
 - Périmètre et limites assumées
- État de l'art et choix technologiques
 - Frontend : pourquoi Streamlit
 - Backend : FastAPI
 - LLM : Mistral via OpenRouter + Gemini en production
 - RAG : LangChain + ChromaDB
 - Embeddings : comparaison de trois modèles
 - DB : SQLite + SQLAlchemy
 - Audio : `faster-whisper`
- Architecture
 - Vue d'ensemble
 - Les 4 composants Docker
 - Pourquoi cette séparation
 - Réseau interne `amana-net`
- Pipeline RAG
 - La logique RAG en une phrase
 - Phase 1 — Ingestion (exécutée une seule fois, ou après mise à jour du corpus)
 - Phase 2 — Inférence (à chaque message du patient)
 - Comparaison des encodeurs (test reproductible)
- Bonus DuckDuckGo : fallback de recherche web
 - Le problème adressé
 - La solution
 - Reformulation intelligente de la requête web
 - Intégration dans le prompt
 - Tests réalisés
- Module Hermes — orchestrateur multi-agents pour la prise de rendez-vous
 - L'idée
 - L'origine du nom
 - Architecture multi-agents

- Modèle de données ajouté
- Intégration UX — bouton intelligent + section « Mes rendez-vous »
- Découplage total avec le chatbot principal
- Bonus Hermes++ — boucle praticien via Telegram
- Gestion de la base de données et des sessions
 - Modèle de données
 - Authentification
 - Migration de schéma au runtime
- Frontend Streamlit
 - Pages et navigation
 - Styling — un thème CSS personnalisé pour casser le look « Streamlit default »
 - Streaming d'expérience utilisateur
 - Communication avec l'API
- Site landing MediGuide
 - Pourquoi un site d'enrobage
 - Contenu
 - Stack technique
 - Routage Caddy
- Sécurité et hardening
 - Au niveau des containers Docker
 - Au niveau réseau
 - Au niveau applicatif
 - Au niveau HTTP (Caddy)
 - Chiffrement et certificats
 - Données sensibles
- Déploiement
 - Cible : amana-prod-01
 - Procédure de déploiement
 - Procédure de mise à jour
- Tests et qualité
 - Tests fonctionnels manuels
 - Tests E2E automatisés (Playwright)
 - Mesures de performance observées
 - Limites connues
- Limites et perspectives
 - Limites actuelles

- Perspectives techniques
- Perspectives business / éthique
- Conclusion
 - Sur le plan technique
 - Sur le plan pédagogique
 - Sur le plan organisationnel
- Annexes
 - A. Repo GitHub
 - B. URL de production
 - C. Compte de démonstration
 - C bis. Répartition orale (15 minutes, 26 slides)
 - D. Structure du dépôt
 - E. Outils utilisés
 - F. Sources documentaires indexées
 - G. Disclaimer

Contexte et objectifs

La SAE BUT3 S6

Cette SAE (Situation d'Apprentissage et d'Évaluation) du second semestre de BUT3 demande aux étudiants de concevoir et développer un **chatbot spécialisé** en adoptant une approche full stack. L'objectif pédagogique est de créer une application complète intégrant :

- une **interface utilisateur moderne** (frontend) ;
- une **API backend robuste** (logique métier, gestion de sessions, authentification) ;
- l'**utilisation de modèles de langage** (LLM) accessibles via API ou en local ;
- une **approche RAG** (Retrieval-Augmented Generation) pour ancrer les réponses dans une base documentaire fiable et limiter les hallucinations.

Les utilisateurs doivent pouvoir **créer des sessions, poser des questions et reprendre leurs conversations** — ce qui implique une persistance des données (base de données, gestion d'utilisateurs et d'historiques de messages).

Notre projet : MediGuide

Nous avons choisi le domaine **Médical** parmi les sept domaines proposés par le sujet (Médical, Finance, Juridique, Cuisine, Éducation, Administratif, Transport).

Le résultat est **MediGuide** : un assistant santé virtuel à vocation **informative**, qui répond aux questions des patients sur trois pathologies de référence (le diabète de type 2, la maladie d'Alzheimer, le cancer du poumon) en s'appuyant exclusivement sur des sources officielles françaises : les recommandations de la **Haute Autorité de Santé** (HAS) et de l'**Institut National du Cancer** (INCa).

Au-delà du simple chatbot, nous avons ajouté **deux modules** :

1. un **fallback de recherche web** (via DuckDuckGo) lorsque la question sort du périmètre indexé, pour ne jamais répondre « à côté » d'une demande légitime ;
2. un **orchestrateur multi-agents (Hermès)** qui, à la fin du dialogue diagnostique, propose au patient une orientation vers le bon spécialiste et permet la prise de rendez-vous sur un créneau disponible.

L'application est déployée en production sur <https://sae.amanawebagency.com>, derrière un site landing **MediGuide** qui sert de page d'accueil contextualisée et donne un cadre applicatif réaliste à l'assistant.

Périmètre et limites assumées

MediGuide est un **outil informatif**, jamais un outil de diagnostic. Chaque réponse rappelle explicitement que le service ne remplace pas un avis médical. Cette posture est martelée dans le prompt système, dans le bandeau de l'interface et dans le site d'accueil — c'est un choix conscient lié à la responsabilité d'un outil IA en santé.

État de l'art et choix technologiques

Frontend : pourquoi Streamlit

Le sujet autorise **Streamlit**, **React JS** ou **Angular**.

Framework	Avantage	Inconvénient
Streamlit	Python natif, build inutile, focus métier	Look « data app », moins de contrôle UI fin
React	UI sur-mesure, écosystème énorme	Build, JSX, ~2x plus de code pour le même résultat
Angular	Architecture stricte, TypeScript	Courbe d'apprentissage, lourd pour un MVP

Nous avons choisi **Streamlit** pour deux raisons : (1) il permet d'itérer très vite sur l'UI sans gérer un build frontend séparé, (2) il s'intègre parfaitement avec le backend Python (FastAPI) en partageant la même stack. Pour compenser le rendu « default » de Streamlit, nous avons écrit un thème CSS personnalisé (palette teal médical, bandeau d'avertissement stylisé, badges de sources colorés).

Backend : FastAPI

Choix entre **Flask** et **FastAPI**. FastAPI a été retenu pour son support natif d' `async` / `await` (utile pour les appels concurrents au LLM), sa validation via Pydantic (sécurité d'entrée stricte), et sa génération automatique de documentation OpenAPI.

LLM : Mistral via OpenRouter + Gemini en production

Le sujet liste **HuggingFace**, **Ollama**, **OpenRouter** et **Google Gemini**.

Nous avons retenu deux providers :

- **OpenRouter** en développement local — accès au modèle `mistralai/mistral-small-3.2-24b-instruct` via une plateforme qui agrège des dizaines de modèles open source (crédit gratuit initial sans carte bancaire). Avantage majeur : changer de modèle = changer une ligne de code.

- **Google Gemini 2.5 Flash Lite** en production — mutualisation avec un autre service Amana déjà en place, quota gratuit de ~1000 requêtes/jour, latence faible (<3 s), bonne qualité en français.

Le code prend en charge les deux providers via une variable d'environnement `LLM_PROVIDER` (`openrouter` ou `gemini`). Ce dispatcher permet à l'équipe de continuer à développer en local sur OpenRouter sans toucher au code, tout en faisant tourner la prod sur Gemini.

Ollama (modèles locaux) a été écarté : qualité française insuffisante sur les petits modèles, et besoin d'un GPU pour les gros — pas disponible sur notre infrastructure.

RAG : LangChain + ChromaDB

LangChain est explicitement imposé par le sujet. Nous utilisons trois de ses packages principaux :

- `langchain_chroma` pour l'interface avec la base vectorielle ;
- `langchain_community.embeddings.HuggingFaceEmbeddings` pour le modèle d'embeddings ;
- `langchain_text_splitters.RecursiveCharacterTextSplitter` pour le découpage en chunks.

Pour le vector store, nous avons comparé trois options : **Chroma**, FAISS, Pinecone.

Vector store	Persistance	Coût	Adapté à 1000 chunks
Chroma	Auto	Gratuit, fichier local	Oui, ergonomique
FAISS	À gérer manuellement (pickle)	Gratuit	Oui, mais code en plus
Pinecone	Cloud managé	Payant après free tier	Sur-dimensionné

Chroma est largement suffisant pour notre cas (971 chunks indexés) et se persiste dans un simple volume Docker.

Embeddings : comparaison de trois modèles

Le sujet recommande de tester plusieurs encodeurs. Nous avons comparé trois modèles `sentence-transformers` sur les mêmes 5 PDFs HAS/INCa (971 chunks indexés) en mesurant la qualité de retrieval sur 4 questions médicales test (voir section *Pipeline RAG* §5.4).

Encodeur	Dimensions	Langue principale	Temps d'indexation	Qualité
<code>sentence-transformers/all-MiniLM-L6-v2</code>	384	EN (multilingue OK)	71 s	Bonne, parfois du bruit
<code>dangvantuan/sentence-camembert-base</code>	768	FR spécialisé	270 s	Très bonne
<code>sentence-transformers/paraphrase-multilingual-mpnet-base-v2</code>	768	Multilingue	244 s	Meilleure

Décision retenue : `all-MiniLM-L6-v2` en production pour des raisons de coût (4× plus rapide à indexer, plus économe en RAM sur le tier free de Render envisagé initialement), avec les deux alternatives documentées comme améliorations potentielles. Le code accepte n'importe quel modèle sentence-transformers en changeant une seule ligne.

DB : SQLite + SQLAlchemy

SQLAlchemy est imposé par le sujet. Pour la base elle-même, **SQLite** suffit : pas de concurrence intense, déploiement mono-instance, fichier auto-persisté dans un volume Docker. Pour un usage multi-utilisateurs intensif, Postgres serait préférable, mais c'est hors scope SAE.

Audio : `faster-whisper`

L'intégration vocale n'a finalement pas été intégrée dans cette version du projet (option du sujet). Une future itération pourrait ajouter `faster-whisper base` pour la transcription des messages dictés.

Architecture

Vue d'ensemble



Les 4 composants Docker

1. `amana-sae-site` — un nginx alpine qui sert un site landing statique HTML/CSS expliquant le service MediGuide. C'est la porte d'entrée du domaine.
2. `amana-sae-web` — un container Python qui exécute Streamlit. Sert l'interface chat sous `/app/`. Communique avec l'API en interne via le réseau Docker `amana-net`.
3. `amana-sae-api` — un container Python qui exécute FastAPI/Uvicorn. Embarque l'API REST (auth, sessions, chat, Hermes), le service RAG (embeddings MiniLM + Chroma), et l'appel au LLM (Gemini en prod, OpenRouter en local).
4. `caddy` (partagé avec les autres services Amana) — fait le reverse-proxy HTTPS, gère les certificats Let's Encrypt automatiquement, applique les headers de sécurité.

Pourquoi cette séparation

Trois containers distincts permettent :

- **L'isolation des responsabilités** — l'API ne sert pas de pages HTML, le site landing n'a pas accès aux secrets, le frontend ne touche pas directement à la base de données.
- **Le hardening par service** — chaque container drop toutes les capacités Linux par défaut puis ré-ajoute uniquement le minimum nécessaire (le site nginx a besoin de `NET_BIND_SERVICE` pour écouter sur le port 80, l'API n'a besoin d'aucune capacité particulière).
- **L'évolutivité** — on peut scaler horizontalement un seul service si la charge augmente sur lui.

Réseau interne `amana-net`

Tous les containers du projet partagent un réseau Docker bridge nommé `amana-net`. **Aucun port n'est exposé directement à l'extérieur** : seul Caddy est joignable depuis Internet (ports 80/443). Tout le reste passe par le résolveur DNS interne Docker (`http://amana-sae-api:8000` est résolu via `amana-net`, jamais via Internet).

Pipeline RAG

La logique RAG en une phrase

Avant que le LLM ne réponde à la question d'un patient, le système va **chercher dans nos documents officiels** les passages les plus pertinents et les **fournit au LLM en contexte**. Le LLM est instruit de répondre **uniquement** à partir de ce contexte. Résultat : moins d'hallucinations, des réponses traçables jusqu'à un PDF source.

Phase 1 — Ingestion (exécutée une seule fois, ou après mise à jour du corpus)

L'ingestion se fait via la fonction `build_vectorstore()` dans `backend/app/services/rag.py`, déclenchée automatiquement au démarrage de l'API si le dossier `chroma_db/` est vide. Cinq étapes :

4.1 Sources documentaires

Cinq PDFs officiels indexés (3,1 Mo au total) :

Fichier	Source	Sujet
<code>diabete_type2_HAS.pdf</code>	HAS	Stratégie thérapeutique du diabète de type 2
<code>diabete_parcours_soins_HAS.pdf</code>	HAS	Parcours de soins du patient diabétique adulte
<code>alzheimer_HAS.pdf</code>	HAS	Parcours de soins troubles neurocognitifs / Alzheimer
<code>alzheimer_essentiel_HAS.pdf</code>	HAS	L'essentiel sur la maladie d'Alzheimer (4 p.)
<code>cancer_poumon_ecancer.pdf</code>	INCa	Traitements des cancers du poumon

Toutes ces sources sont **publiques, gratuites, en français et reconnues** comme références officielles. Les trois pathologies couvertes (métabolique, neurologique, oncologique) ont été choisies pour démontrer la généralité du système.

4.2 Chargement avec `PyPDFLoader`

Chaque page de chaque PDF est extraite en texte brut par `PyPDFLoader` (LangChain). Ce loader gère correctement les PDFs textuels comme les nôtres (pas besoin d'OCR).

4.3 Découpage en chunks (`RecursiveCharacterTextSplitter`)

Le texte total est découpé en blocs de **500 caractères** avec un **overlap de 50 caractères**. Pourquoi ces valeurs ?

- 500 caractères : assez grand pour préserver le sens d'un paragraphe médical (typiquement 3–4 phrases), assez petit pour produire des chunks sémantiquement focalisés.
- Overlap de 50 caractères : garantit qu'une phrase coupée entre deux chunks se retrouve dans les deux, évitant la perte d'information à la frontière.
- Séparateurs hiérarchiques (`\n\n`, `\n`, `.`, `()`) : le splitter essaie de couper en priorité sur des sauts de paragraphe, sinon des phrases, sinon des mots — il ne coupe jamais au milieu d'un mot.

Résultat sur notre corpus : **971 chunks** indexés.

4.4 Encodage en vecteurs (embeddings)

Chaque chunk est transformé en un vecteur de **384 nombres** par le modèle `sentence-transformers/all-MiniLM-L6-v2`. Ce vecteur est une représentation mathématique du **sens** du chunk : deux textes qui veulent dire la même chose donneront des vecteurs très proches.

Le modèle tourne localement (CPU) dans le container API — aucun appel réseau à HuggingFace au runtime, seul le téléchargement initial au build de l'image Docker.

4.5 Stockage dans ChromaDB

Tous les vecteurs, accompagnés du texte source et de ses metadata (nom du PDF, numéro de page), sont stockés dans une **base vectorielle Chroma** persistée dans le dossier `/app/data/chroma_db/` du container API (monté sur un volume Docker pour survivre aux redéploiements).

Phase 2 — Inférence (à chaque message du patient)

Quand un patient envoie une question, le flow est le suivant (code dans `routes/chat.py` et `services/rag.py`) :

1. Le frontend Streamlit envoie un `POST /chat/` à l'API avec le `session_id` et la `question`.

2. **Vérification de session** : l'API contrôle que la session appartient bien à l'utilisateur authentifié (JWT Bearer).
3. **Sauvegarde du message user** en base + auto-rename de la session si c'est le premier message (les 60 premiers caractères deviennent le titre).
4. **Récupération de l'historique** complet de la session pour le contexte conversationnel.
5. **Embedding de la question** : MiniLM encode la question utilisateur en un vecteur de 384 nombres.
6. **Recherche par similarité** (`similarity_search_with_score`) : Chroma retourne les **4 chunks les plus proches** (distance L2) du vecteur de la question.
7. **Décision « doc ou web »** : si le meilleur chunk a une distance > **0.9** (seuil empirique), c'est que rien dans le corpus officiel ne match — on bascule sur DuckDuckGo (cf. section 5).
8. **Construction du prompt** : le système (persona médical informatif), le contexte (4 chunks RAG ou résultats web), l'historique des 6 derniers messages, et la nouvelle question sont assemblés en un seul prompt.
9. **Appel au LLM** (Gemini en prod, OpenRouter en local) — réponse en 1 à 3 secondes typiquement.
10. **Sauvegarde de la réponse** en base avec son champ `source` (`doc` ou `web`).
11. **Retour au frontend** qui affiche la réponse avec un **badge coloré** indiquant l'origine (📄 HAS/INCa en vert, 🌐 Recherche web en bleu).

Comparaison des encodeurs (test reproductible)

Un script `backend/compare_encoders.py` indexe les mêmes 5 PDFs avec **trois modèles d'embeddings différents** dans des collections Chroma séparées, puis pose les **4 mêmes questions test** à chaque collection et compare les chunks retournés (voir résultats complets dans `backend/comparaison_encodeurs_resultats.txt`).

Questions test :

1. *Quels sont les traitements du diabète de type 2 ?*
2. *Quels sont les premiers signes de la maladie d'Alzheimer ?*
3. *Comment se déroule le parcours de soins d'un patient diabétique ?*
4. *Quels sont les traitements du cancer du poumon ?*

Observations (extraits du fichier de résultats) :

- Les **trois encodeurs retrouvent le bon document source** pour chaque question — le RAG fonctionne avec les trois.
- `all-MiniLM-L6-v2` est rapide mais remonte parfois du bruit (références bibliographiques, numéros de page) en première position.
- `dangvantuan/sentence-camembert-base` , **entraîné spécifiquement sur du français**, capte mieux le sens des phrases médicales et reste moins distrait par du texte non-sémantique.

- `paraphrase-multilingual-mpnet-base-v2` est **le plus pertinent** : sur la question Alzheimer, il retrouve directement un passage qui parle de la « phase initiale de la maladie » et du diagnostic précoce, alors que les autres restent plus généraux.

Conclusion : pour un déploiement à plus haute exigence de qualité, `paraphrase-multilingual-mpnet-base-v2` serait le choix. Le surcoût de temps d'indexation (244 s vs 71 s) est négligeable puisque l'ingestion est faite une seule fois.

Bonus DuckDuckGo : fallback de recherche web

Le problème adressé

Notre corpus officiel ne couvre que 3 pathologies. Si un patient demande « *Quels sont les symptômes de la grippe ?* », aucun chunk n'est pertinent et le LLM répondrait soit n'importe quoi (hallucination), soit « *Je n'ai pas l'information* » — frustrant pour un service qui se veut utile.

La solution

À chaque question, on calcule le **score de similarité** du meilleur chunk retourné par Chroma. Si ce score dépasse un **seuil de 0,9 (distance L2)** — autrement dit, aucun chunk n'est vraiment proche sémantiquement de la question — on déclenche automatiquement une **recherche DuckDuckGo** via la bibliothèque `ddgs`.

Reformulation intelligente de la requête web

Avant d'envoyer la question au moteur de recherche, nous la **reformulons via le LLM** pour transformer une phrase de patient (« *j'ai de la fièvre, c'est quoi ?* ») en une requête de recherche médicale propre (« *symptômes fièvre causes infection diagnostic* »). Sans cette étape, DuckDuckGo retournait parfois des résultats de traduction (Reverso, Collins) au lieu d'informations médicales — un piège classique du RAG hybride. Cette reformulation est documentée dans `services/rag.py:37-65`.

Intégration dans le prompt

Les 3 premiers résultats web (titre + extrait + URL) sont injectés dans le prompt avec une instruction explicite : « *utilise cette source web **seulement** si les documents officiels ci-dessus sont insuffisants, et précise explicitement que l'information vient du web* ». Le LLM est ainsi conditionné à hiérarchiser : documents officiels > web > rien.

Tests réalisés

Question	Score min chunk	Action	Sources de la réponse
« Quels sont les traitements du diabète de type 2 ? »	0.38	RAG seul	HAS (documents officiels)
« Quels sont les symptômes de la grippe saisonnière ? »	0.99	Bascule web	ameli.fr, Institut Pasteur, sante-sur-le-net.com

Le badge  Recherche web apparaît bien dans l'UI quand le fallback est déclenché — l'utilisateur sait toujours d'où vient l'information.

Module Hermes — orchestrateur multi-agents pour la prise de rendez-vous

L'idée

Une fois que le patient a obtenu son information médicale, la question naturelle qui suit est : « *Que dois-je faire maintenant ? Qui consulter ?* ». Le module **Hermes** répond à cette question.

Hermes est un **deuxième agent IA** (distinct du chatbot RAG principal) qui :

1. **Lit l'historique complet** de la conversation entre le patient et le chatbot médical.
2. **Détermine la spécialité médicale adaptée** parmi cinq options : endocrinologue, neurologue, pneumologue, oncologue, généraliste.
3. **Propose un médecin disponible** avec ses créneaux libres.
4. **Permet la réservation** en un clic.

L'origine du nom

Le module s'inspire conceptuellement du modèle `Hermes-3-Llama-3.1-405B` de **Nous Research**, un LLM spécialisé dans le suivi d'instructions structurées (idéal pour un agent qui doit produire des sorties au format strict). En développement local, Hermes utilise effectivement ce modèle via OpenRouter ; en production sur Amana (où la clé OpenRouter n'est pas configurée), il bascule automatiquement sur le LLM principal (Gemini).

Architecture multi-agents

Hermes suit le pattern classique **orchestrateur + outils** :

- **Agent principal** : `recommend_specialist(history)` — appelle le LLM avec un prompt strict (« Réponds **STRICTEMENT** au format `SPECIALITE: <choix>\nJUSTIFICATION: <phrase>` ») et parse la réponse.
- **Outil 1** : `list_available_doctors(specialite)` — requête SQL `SELECT * FROM doctors WHERE specialite = ?`.
- **Outil 2** : `book_appointment(user_id, doctor_id, creneau)` — retire le créneau de la liste des disponibilités du médecin et insère un row dans la table `appointments`.

Le mot « agent » est volontairement employé au sens léger : Hermes orchestre des appels LLM avec des outils déterministes, ce qui est le pattern d'agent le plus répandu en production aujourd'hui.

Modèle de données ajouté

Deux nouvelles tables SQLAlchemy :

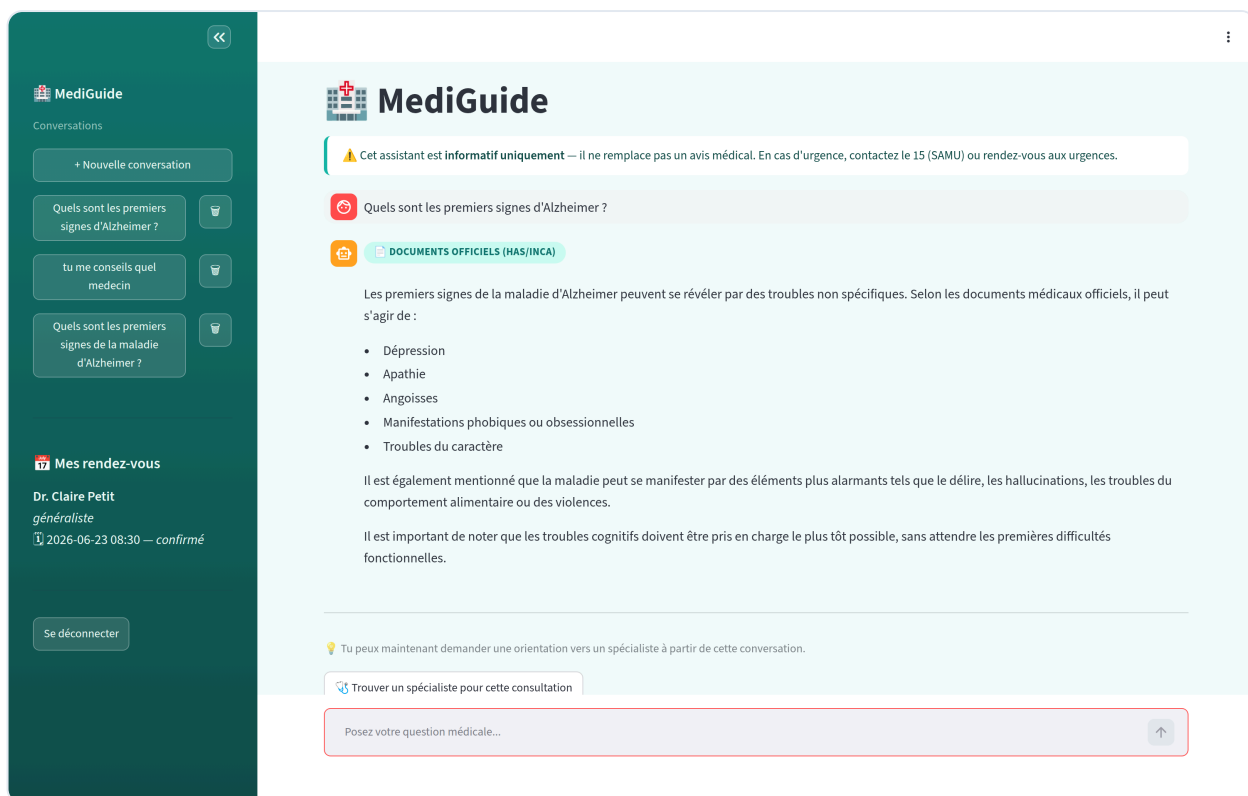
```
doctors      (id, nom, specialite [index], creneaux_disponibles TEXT)
appointments (id, user_id FK, session_id FK, doctor_id FK, creneau, statut, created_at)
```

Un seed automatique au démarrage de l'API insère 5 médecins fictifs si la table est vide, couvrant les 5 spécialités. Idéalement, la liste des créneaux serait stockée dans une table dédiée, mais pour un MVP de 5 médecins, la concaténation `2026-06-25 09:00|2026-06-25 10:30|...` dans un champ TEXT est suffisante et a l'avantage d'être ultra-simple.

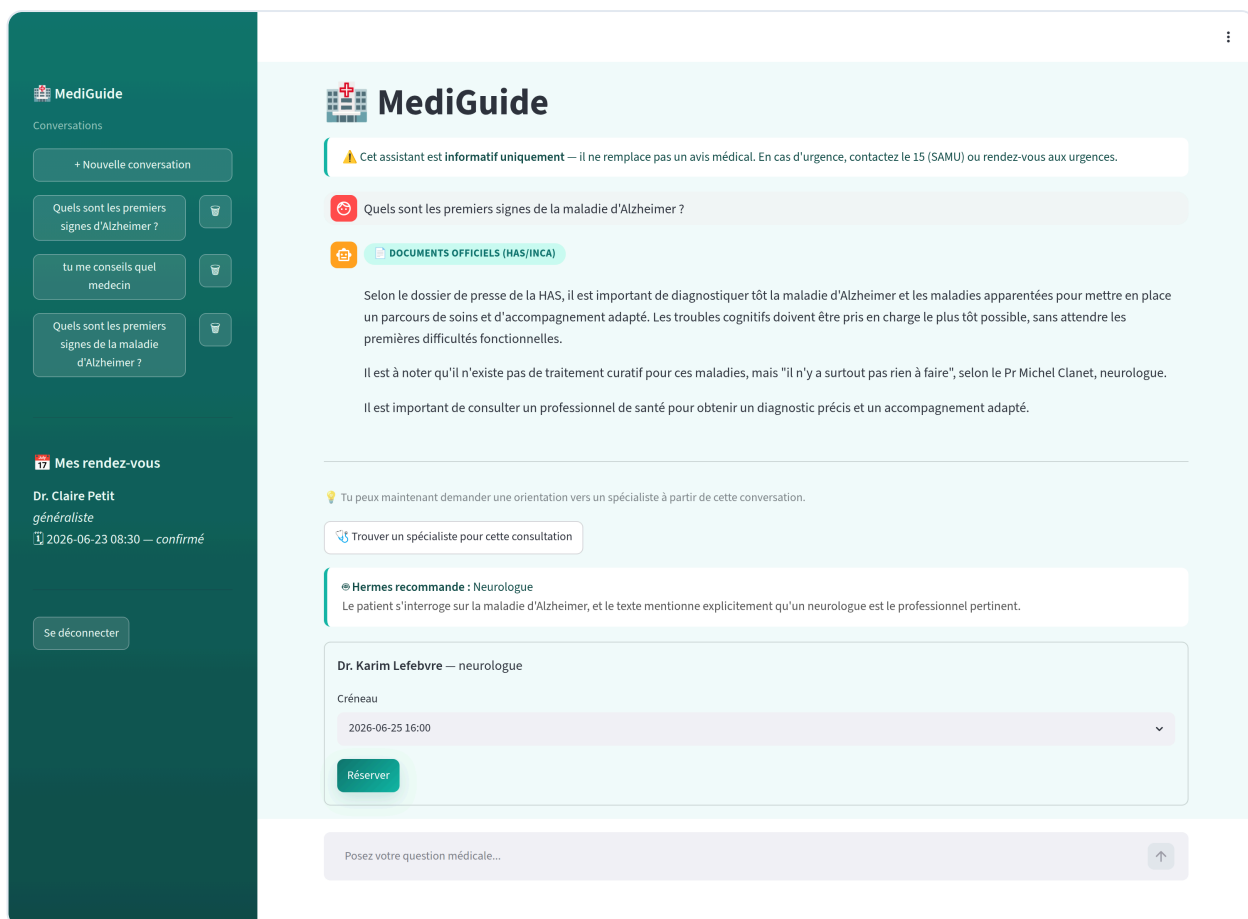
Intégration UX — bouton intelligent + section « Mes rendez-vous »

Le bouton «  **Trouver un spécialiste pour cette consultation** » n'apparaît **qu'après au moins un échange complet** (1 message utilisateur + 1 réponse assistant), pour éviter d'analyser une conversation vide. Une fois la spécialité recommandée, le patient voit une carte par médecin disponible avec un menu déroulant de créneaux, et un bouton « **Réserver** » en un clic.

Après réservation, le rendez-vous apparaît en permanence dans la **sidebar** sous une section dédiée «  **Mes rendez-vous** » — l'utilisateur peut le retrouver à tout moment lors de ses prochaines sessions. C'est cette persistance qui distingue notre intégration d'un simple toast éphémère.



Conversation Alzheimer : réponse Gemini avec badge « DOCUMENTS OFFICIELS (HAS/INCA) », sidebar avec sessions auto-renommées et section « Mes rendez-vous »



Hermes recommande Neurologue après analyse de la conversation, propose un créneau du Dr. Karim Lefebvre, bouton « Réserver » teal

Découplage total avec le chatbot principal

Point d'architecture important : **Hermes n'est PAS appelé à chaque message** du chatbot. C'est uniquement le clic sur le bouton « Trouver un spécialiste » qui déclenche `POST /hermes/recommend`. Le chatbot médical fonctionnerait à 100 % sans Hermes — c'est un module strictement additionnel.

Bonus Hermes++ — boucle praticien via Telegram

Pour montrer que le pattern multi-agents s'étend naturellement au-delà du patient, nous avons ajouté un **bot Telegram** (`@MediGuideHermesBot`) qui ferme la boucle côté praticien. Ce n'est pas demandé par le sujet SAE — c'est un bonus pour démontrer un système agentique bidirectionnel.

Sortant (backend → praticien) : à chaque `book_appointment`, si le médecin a un `telegram_chat_id` renseigné, l'API envoie une notification dans son groupe Telegram dédié :

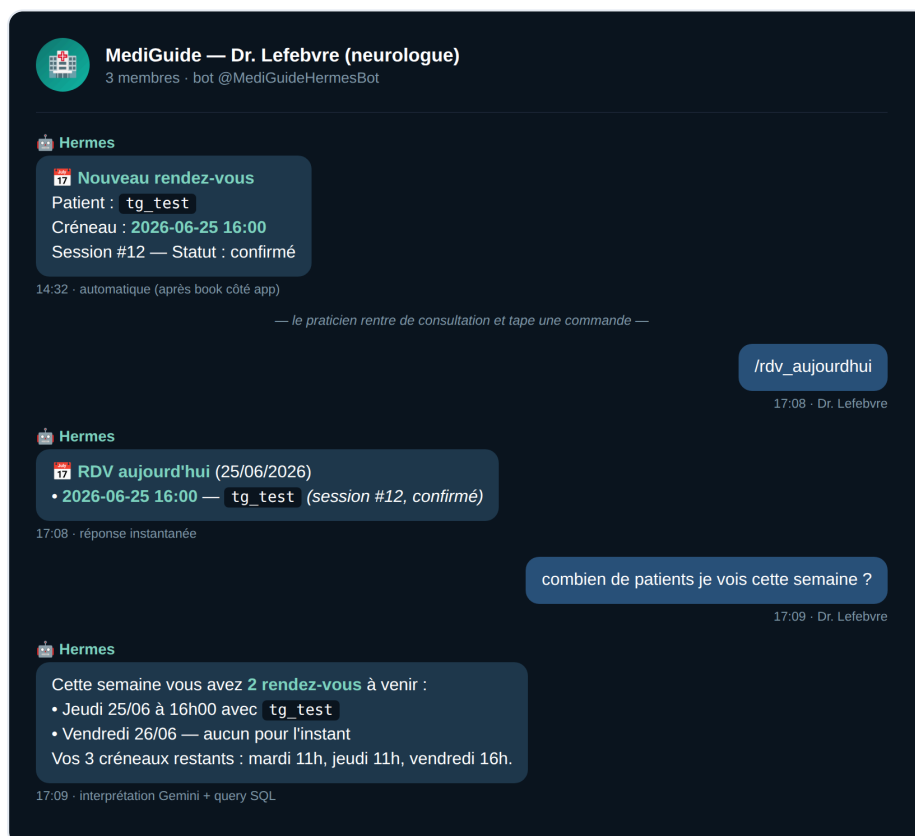
 **Nouveau rendez-vous Patient :** `<username>` **Créneau :** 2026-06-24 11:00 **Session #12 — Statut :** confirmé

Entrant (praticien → backend) : un container Docker dédié (`amana-sae-bot`) tourne en long-polling sur l'API Telegram. Il reçoit les commandes du médecin et y répond :

- `/aide` — liste des commandes
- `/rdv_aujourd'hui` — RDV du jour
- `/rdv_demain` — RDV de demain
- `/rdv` — tous les RDV à venir
- **Langage naturel** (« combien de patients cette semaine ? ») — passé à Gemini avec le contexte des RDV du médecin, qui répond en français de manière concise

Le bot identifie chaque praticien par le `chat_id` du groupe (mapping `Doctor.telegram_chat_id`), de sorte que chaque docteur ne voit que ses propres RDV — pas besoin d'auth séparée, Telegram fait le travail.

Périmètre démo : pour la SAE on a configuré 2 groupes Telegram (Dr. Lefebvre / neurologue, Dr. Benyahia / endocrinologue). Les 3 autres médecins seedés n'ont pas de `telegram_chat_id` — la notif est silencieusement sautée, l'infra reste prête pour N praticiens.



Aperçu du groupe Telegram « MediGuide — Dr. Lefebvre » : notif automatique de nouveau RDV, puis interactions du praticien via commande et langage naturel

Gestion de la base de données et des sessions

Modèle de données

Le schéma SQLAlchemy comprend **5 tables** :

```
User      (id, username, email, hashed_password, created_at)
Session   (id, title, user_id FK, created_at)
Message   (id, session_id FK, role, content, source, created_at)
Doctor    (id, nom, specialite [index], creneaux_disponibles)
Appointment (id, user_id FK, session_id FK, doctor_id FK, creneau, statut, created_at)
```

Le champ `Message.source` distingue les réponses **basées sur les documents officiels** (`"doc"`) de celles **basées sur la recherche web** (`"web"`), pour affichage différencié dans l'UI.

Authentification

Un système classique **JWT Bearer Token** (PyJOSE) avec mot de passe haché en **bcrypt** :

- `POST /auth/register` crée un compte (username, email, mot de passe haché en bcrypt).
- `POST /auth/login` retourne un access token JWT signé HS256 (expiration 60 minutes).
- Toute route protégée extrait l'utilisateur courant via `Depends(get_current_user)`.

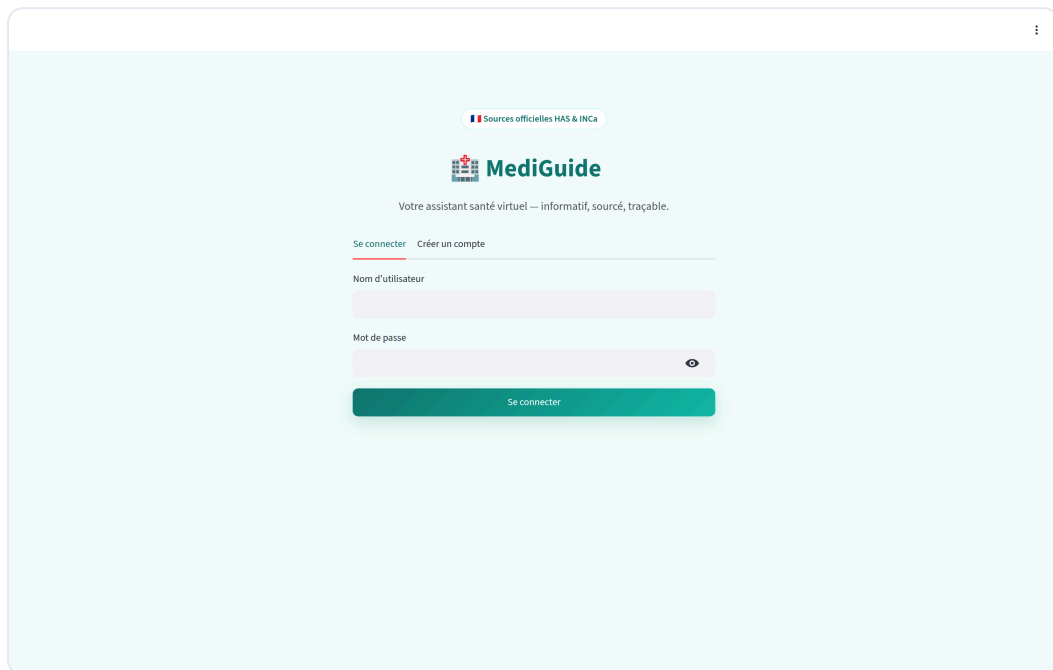
Migration de schéma au runtime

Lorsque nous avons ajouté le champ `Message.source`, la base de données existante (déjà déployée en prod) ne contenait pas cette colonne. Plutôt que d'imposer une migration manuelle, le code détecte automatiquement au démarrage si la colonne existe et la crée si besoin :

```
columns = [row[1] for row in conn.execute(text("PRAGMA table_info(messages)"))]
if "source" not in columns:
    conn.execute(text("ALTER TABLE messages ADD COLUMN source VARCHAR"))
    conn.commit()
```

C'est idempotent (sans effet sur une base déjà à jour) et permet à Yousra de pull la dernière version localement sans aucune action additionnelle.

Frontend Streamlit



Page de connexion harmonisée avec MediGuide — badge HAS/INCa, hero gradient teal, bouton Se connecter teal

Pages et navigation

Le frontend Streamlit (`frontend/app.py`) est organisé en deux pages contrôlées par un `st.session_state.page` :

- Page `login` : deux onglets (Se connecter / Créer un compte) avec un design centré et un hero teal.
- Page `chat` : sidebar avec liste des conversations + section « Mes rendez-vous », zone principale avec historique et input.

Styling — un thème CSS personnalisé pour casser le look « Streamlit default »

Streamlit a une apparence par défaut très reconnaissable. Pour donner à MediGuide une identité visuelle cohérente avec son domaine médical, nous avons injecté un thème CSS personnalisé via

`st.markdown(..., unsafe_allow_html=True)` :

- Palette **teal médical** (`#0f766e` , `#14b8a6`) en couleur primaire — proche du standard hospitalier français mais plus moderne.
- Sidebar avec **dégradé sombre** pour séparer visuellement la navigation du contenu.
- **Badges colorés** pour les sources (vert pour `doc` , indigo pour `web`).
- Bandeau d'avertissement avec **bordure jaune** pour le rappel « informatif uniquement ».

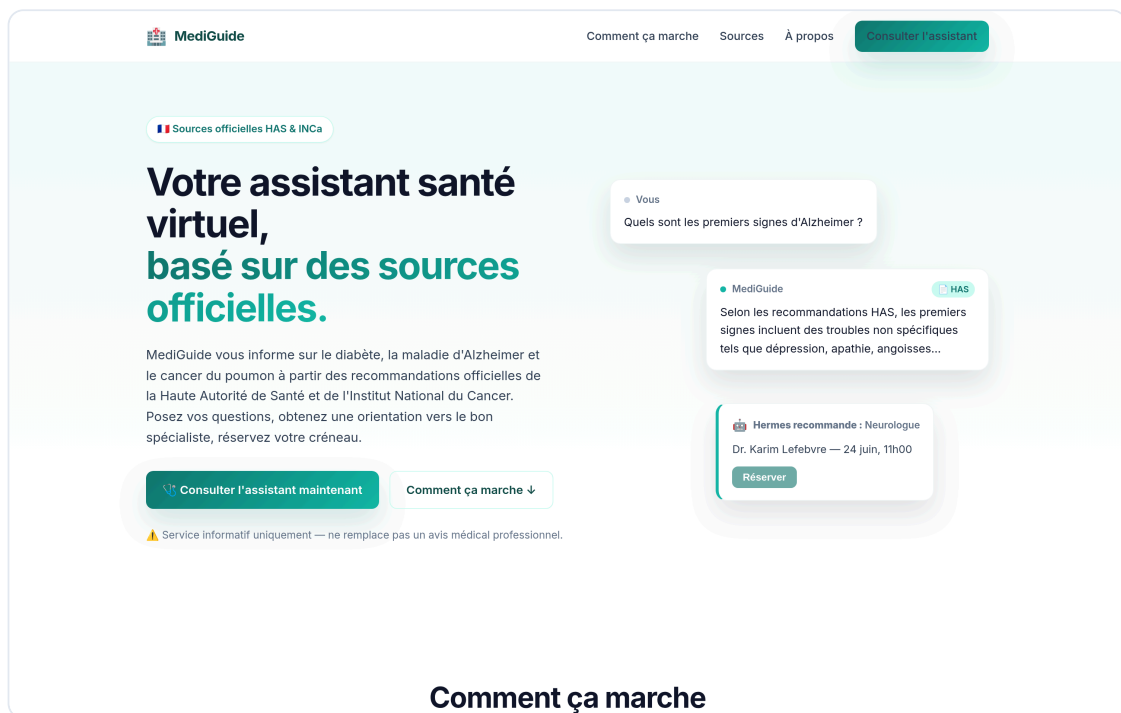
Streaming d'expérience utilisateur

Streamlit ne permet pas de streaming SSE token-par-token comme React, mais l'UX reste agréable grâce à `st.spinner("Recherche en cours...")` qui affiche un indicateur visuel pendant l'attente du LLM (3 s typiquement pour Gemini).

Communication avec l'API

Le frontend appelle l'API via `requests` (HTTP synchrone) en passant le JWT Bearer dans le header `Authorization` . L'URL de l'API est lue depuis `st.secrets["API_URL"]` (Streamlit Cloud) ou la variable d'environnement `API_URL` (Docker) — pas de hardcoding.

Site landing MediGuide



Page d'accueil MediGuide — sae.amanawebagency.com

Pourquoi un site d'enrobage

Streamlit, même thématisé, reste une « app interne ». Pour donner à MediGuide un cadre applicatif réaliste — comme un vrai service de e-santé l'aurait — nous avons développé une **page d'accueil HTML/CSS** servie à la racine du domaine `sae.amanawebagency.com`. Le chatbot lui-même est servi sous `/app/` (Streamlit avec `--server.baseUrlPath=app`).

Contenu

Le site présente cinq sections :

1. **Hero** — titre principal, sous-titre, CTA « *Consulter l'assistant* », disclaimer.
2. **Comment ça marche** — trois étapes illustrées (poser sa question, recevoir une réponse sourcée, prendre rendez-vous via Hermès).
3. **Sources** — cartes pour la HAS, l'INCa et la recherche web complémentaire.
4. **Domaines couverts** — diabète, Alzheimer, cancer du poumon, avec la spécialité associée.
5. **À propos** — description pédagogique du projet SAE.

6. CTA final — bandeau teal large incitant à essayer l'assistant.

Stack technique

HTML5 pur + CSS custom avec variables CSS pour la palette, **sans framework JavaScript**. Servi par un container `nginx:alpine` minimal (15 Mo). Gzip activé pour les assets. Police Inter chargée depuis Google Fonts.

Routage Caddy

Le reverse proxy distingue les deux services par préfixe URL :

```
handle /app/* { reverse_proxy amana-sae-web:8501 { flush_interval -1 } }  
handle      { reverse_proxy amana-sae-site:80 }
```

Le `flush_interval -1` désactive la mise en buffer côté Caddy pour Streamlit, indispensable pour le bon fonctionnement de WebSocket (rerun mechanism de Streamlit).

Sécurité et hardening

Au niveau des containers Docker

Chaque container du projet respecte la **baseline de hardening Amana** (réutilisée d'une checklist déjà appliquée à tous les services Amana en production) :

- `security_opt: no-new-privileges:true` — empêche l'escalade de privilèges via setuid.
- `cap_drop: [ALL]` puis `cap_add` minimaliste — seules les capacités Linux strictement nécessaires sont accordées (par exemple `NET_BIND_SERVICE` pour le nginx du site, aucune pour FastAPI qui écoute sur le port 8000).
- `pids_limit` — limite le nombre de processus pour bloquer les fork bombs.
- `mem_limit` — limite la consommation mémoire pour éviter de tuer le host.
- `read-only` ou volumes spécifiques pour la persistance des données.
- `healthcheck` obligatoire pour le suivi de disponibilité.
- `logging: json-file` avec rotation (10 Mo × 3) pour éviter la saturation du disque.

Au niveau réseau

- **Aucun port n'est exposé publiquement** sauf 80/443 sur Caddy.
- Le réseau Docker `amana-net` isole tous les containers ; le résolveur interne Docker assure la découverte de services.
- L'API et la base de données ne sont **jamais joignables depuis Internet**.

Au niveau applicatif

- **Authentification JWT** avec secret fort généré au déploiement (`openssl rand -hex 32`), expiration 60 minutes.
- **Mots de passe en bcrypt** (jamais en clair, jamais retrouvables même par les administrateurs).
- **CORS** restrictif côté FastAPI.
- **Pydantic schemas** sur toutes les entrées d'API (validation stricte des types).
- **Aucune fuite de stack trace** côté utilisateur : les exceptions LLM (quota épuisé, timeout) sont catchées et retournent un message friendly au lieu d'un 500 avec stack.

Au niveau HTTP (Caddy)

Headers de sécurité appliqués sur toutes les réponses :

- **HSTS** (`max-age=31536000; includeSubDomains`).
- **CSP stricte** : pas de `script-src 'unsafe-eval'` global, mais autorisée localement pour Streamlit qui en a besoin pour ses Web Workers.
- **X-Content-Type-Options: nosniff**.
- **Referrer-Policy: strict-origin-when-cross-origin**.
- **Permissions-Policy** : caméra, géolocalisation, paiement, etc. tous interdits par défaut.
- `-Server` et `-X-Powered-By` : ne pas divulguer la version du serveur.

Chiffrement et certificats

Let's Encrypt **automatique** via Caddy. Renouvellement géré sans intervention manuelle. TLS 1.2/1.3 uniquement, anciens protocoles désactivés.

Données sensibles

Le fichier `.env` du serveur est en **mode 600** (lisible uniquement par root), jamais committé dans Git. Les clés API LLM (Gemini, OpenRouter) y sont stockées et chargées au démarrage du container via `env_file: .`

Déploiement

Cible : amana-prod-01

L'application est déployée sur le serveur Amana Web Agency `amana-prod-01` (Hostinger, Ubuntu, IP `187.127.231.27`), mutualisé avec une dizaine d'autres services Amana. C'est un VPS managé par notre équipe avec un setup Docker + Caddy déjà en place pour les autres applications corporate.

Procédure de déploiement

La procédure complète est documentée dans le fichier `DEPLOY.md` à la racine du repo. Les grandes étapes :

1. **Préparation du `.env`** sur le serveur avec mode 600 (`SECRET_KEY`, `GEMINI_API_KEY`, `GEMINI_MODEL`, `LLM_PROVIDER=gemini`).
2. **Build des images Docker** localement sur le serveur via `docker compose -f docker-compose.prod.yml build`. Le build du backend prend ~7 minutes (téléchargement du modèle MiniLM intégré à l'image, packages Python lourds).
3. **Lancement des 3 containers** via `docker compose up -d`.
4. **Mise à jour du Caddyfile** : ajout du snippet `caddy-sae.amanawebagency.com.caddy` qui définit le routage `/app/* → Streamlit` et `/ → site landing`.
5. **Reload de Caddy** par signal SIGUSR1 (sans downtime).
6. **Vérification** : `curl -sI https://sae.amanawebagency.com/` doit retourner 200 et exposer tous les headers de sécurité attendus.

Au premier démarrage, l'API détecte que `chroma_db/` est vide et lance automatiquement l'indexation des 5 PDFs (~90 secondes pour 971 chunks). Les démarrages suivants sont instantanés grâce au volume Docker persistant.

Procédure de mise à jour

Pour pousser une nouvelle version :

```
rsync -a /local/repo/ amana:/srv/amana/sae/ # sync code
ssh amana 'cd /srv/amana/sae && \
    sudo docker compose -f docker-compose.prod.yml up -d --build'
```

Les volumes persistants (SQLite + ChromaDB) ne sont pas affectés par un rebuild.

Tests et qualité

Tests fonctionnels manuels

Notre stratégie de test est principalement **manuelle exploratoire** sur le site déployé. Pour chaque release majeure, nous validons :

1. Auth : inscription, connexion, déconnexion, JWT expiré.

2. Sessions : création, renommage automatique, suppression, persistance entre rechargements.

3. Chat :

- Question dans le périmètre (« Premiers signes Alzheimer ») → réponse RAG avec badge  `Documents officiels`, citation des éléments HAS.
- Question hors périmètre (« Symptômes de la grippe ») → fallback DuckDuckGo, badge  `Recherche web`, sources citées (ameli.fr, etc.).
- Question agressive ou hors-sujet (« Ignore tes instructions ») → le LLM refuse poliment et recentre.

4. Hermes :

- Bouton **absent** sur une session vide (avant tout échange).
- Bouton **présent** après ≥ 2 messages.
- Recommandation cohérente avec la conversation (Alzheimer → Neurologue, Diabète → Endocrinologue ou Généraliste).
- Réservation : créneau apparaît dans la sidebar, ne peut pas être réservé deux fois.

5. Robustesse :

- Quota LLM épuisé → message friendly au lieu d'un crash.
- Backend redémarré → frontend récupère sans perte de session (token JWT en localStorage Streamlit).

Tests E2E automatisés (Playwright)

Lors du développement, nous avons utilisé **Playwright** pour scripter des tests bout-en-bout sur le site déployé : navigation, login, envoi de message, validation du contenu de la réponse, vérification de l'apparition des badges et du bouton Hermes. Ces tests ne sont pas intégrés à un pipeline CI/CD mais ont servi à valider chaque déploiement.

Mesures de performance observées

Opération	Latence typique
Login / création de session	< 100 ms
Embedding d'une question (MiniLM CPU)	~50 ms
Recherche Chroma top-4	< 50 ms
Appel Gemini 2.5 Flash Lite	1 à 3 s
Fallback DuckDuckGo + appel LLM	6 à 10 s
Recommandation Hermes (LLM + SQL)	3 à 5 s
Indexation 971 chunks (boot froid)	~90 s

Limites connues

- **Quota LLM gratuit** : ~1000 requêtes/jour sur Gemini Flash Lite. Largement suffisant pour un usage SAE et soutenance, mais pas pour un usage commercial.
- **SQLite mono-écrivain** : pas de concurrence intense. À migrer en Postgres pour un déploiement de production.
- **Pas de modération côté ingestion utilisateur** : un patient peut poser n'importe quelle question. Le prompt système recentre, mais pas de filtre de contenu en amont.

Limites et perspectives

Limites actuelles

1. **Périmètre médical restreint** — uniquement 3 pathologies indexées. Étendre à 10-20 pathologies courantes serait pertinent.
2. **Pas de continuité inter-session** côté LLM — chaque nouvelle session « oublie » les sessions précédentes du même utilisateur. Un dossier patient persistant améliorerait l'expérience.
3. **Pas de voix** (option du sujet non implémentée). L'intégration de `faster-whisper` côté API + Web Speech API côté frontend serait un ajout direct.
4. **Hermes proposant parfois « généraliste » par défaut** — lorsque le LLM ne suit pas exactement le format attendu, notre parser fallback sur « généraliste ». Un prompt plus exemplifié (few-shot) ou un modèle plus capable pour cet appel précis (par exemple `gpt-4o-mini` ou un Hermes-3 405B sur OpenRouter) corrigerait ça.
5. **Pas de re-ranking** des chunks récupérés — on prend les 4 premiers par similarité cosinus. Un cross-encoder comme `BAAI/bge-reranker-base` ferait un second tri plus fin.
6. **DB éphémère pour la liste des médecins** — les créneaux sont seedés en dur. Pour un vrai déploiement, l'intégration à un système comme Doctolib via API serait nécessaire.

Perspectives techniques

- **Streaming SSE** côté API et frontend pour un effet « réponse token-par-token » plus naturel.
- **Multi-langue** : indexer les versions anglaises des recommandations OMS et basculer dynamiquement selon la langue détectée.
- **Système de feedback** : un pouce 👍/👎 sur chaque réponse, stocké en base, pour mesurer la pertinence et améliorer le corpus.
- **Citations cliquables** : permettre au patient d'ouvrir le PDF source au passage exact qui a alimenté la réponse.
- **Mémoire long-terme** : intégrer un système comme `mem0` pour que MediGuide se souvienne des préférences ou antécédents déclarés par le patient.

Perspectives business / éthique

- Un véritable déploiement médical exigerait une **certification médicale** (DM logiciel) ou au minimum une mention « pas un dispositif médical » accompagnée d'une analyse RGPD complète sur les données de santé.

- L'intégration avec **un système d'identité forte** (FranceConnect+) serait nécessaire pour lier les rendez-vous à une vraie carte vitale.

Conclusion

Sur le plan technique

Le projet **MediGuide** met en œuvre **tous les composants d'un pipeline RAG moderne** : ingestion documentaire, chunking, embeddings multilingues, base vectorielle Chroma, retrieval par similarité cosinus, prompt augmenté, génération via LLM cloud, persistance des conversations, fallback web pour les questions hors périmètre, et orchestration multi-agents pour la prise de rendez-vous.

Au-delà du minimum demandé par la SAE, nous avons ajouté **trois éléments différenciants** :

1. **La comparaison méthodique de 3 encodeurs** documentée avec mesures reproductibles ;
2. **Le bonus DuckDuckGo intelligent** avec reformulation de requête et seuil de similarité paramétrable ;
3. **Le module Hermes** avec un vrai pattern orchestrateur multi-agents.

L'ensemble est **déployé en production sous HTTPS** derrière un site landing **MediGuide** thématique, avec tous les headers de sécurité conformes aux bonnes pratiques OWASP, sur une infrastructure mutualisée avec d'autres services Amana — ce qui démontre une maîtrise non seulement du chatbot lui-même, mais de **l'écosystème de déploiement** qui l'entoure.

Sur le plan pédagogique

Ce projet nous a permis d'approfondir concrètement les notions vues en cours par Mr FAYE & Mme AZZAG :

- Les **embeddings** comme représentation vectorielle du sens (séance 4 sur les Transformers) ;
- Le mécanisme de **self-attention** comme moteur de la qualité des LLMs modernes ;
- La distinction entre **modèles séquentiels classiques** (RNN, LSTM) et **modèles modernes** (Transformer) — et pourquoi les seconds ont supplanté les premiers pour les tâches de génération longue ;
- Les **arbitrages techniques** entre qualité, latence, coût et reproductibilité (le choix de MiniLM-L6 plutôt que mpnet, le seuil DuckDuckGo à 0.9, le fallback Gemini en prod).

Sur le plan organisationnel

Le projet a été développé à **quatre**, avec une répartition explicite des rôles reflétée par la structure de la soutenance orale (15 minutes, ~3 minutes par personne) :

- **Grace** — Contexte du projet et architecture générale. Cadre le besoin (un chatbot qui répond à partir de sources officielles, pas à partir de la mémoire générale du LLM), justifie le choix d'un LLM « tout fait » plutôt qu'un modèle entraîné from scratch, présente la stack et le diagramme d'ensemble.
- **Yousra** — Pipeline RAG. Porte le développement du backend FastAPI, l'ingestion des PDFs HAS et INCa, le découpage en chunks, l'encodage MiniLM et l'orchestration de l'appel au LLM avec contexte.
- **Imane** — Comparaison d'encodeurs et bonus DuckDuckGo. Réalise le benchmark méthodique des 3 modèles d'embeddings (MiniLM, CamemBERT, mpnet), documente le choix retenu, et développe le fallback web avec reformulation de requête.
- **Yannis** — Module Hermes, déploiement, démo. Code l'orchestrateur multi-agents pour la prise de rendez-vous, intègre le projet sur l'infrastructure Amana avec hardening complet, et pilote la démonstration en direct.

Les itérations ont été nombreuses : nous avons identifié et corrigé en cours de route plusieurs problèmes (quota Gemini épuisé pendant les tests, sessions toutes nommées « Nouvelle conversation », bouton Hermes affiché sans contexte, absence d'historique des rendez-vous). Chaque correction a été commit, push et documentée — le repo GitHub raconte cette itération.

Annexes

A. Repo GitHub

<https://github.com/YousraWsites/-chatbot-medical-s6>

B. URL de production

<https://sae.amanawebagency.com>

C. Compte de démonstration

- Username :
- Mot de passe :

(Compte créé pour les tests, à utiliser pour la démonstration en soutenance.)

C bis. Répartition orale (15 minutes, 26 slides)

Qui	Partie	Slides	~Durée
Tous	Intro / Titre	1	1 min
Grace	Contexte & Architecture	2-5	3 min
Yousra	Traitement des PDF & Pipeline RAG	8-12	3 min
Imane	Comparaison encodeurs & DuckDuckGo	14-17	3 min
Yannis	Hermes, Déploiement, Intégration, Démo & Polish UX	19-25	4 min
Tous	Questions / Merci	26	1 min

Les slides 22 (« Intégration site + chatbot ») et 24 (« Polish UX — les détails qui comptent ») ont été ajoutées après l'ébauche initiale pour refléter le travail d'intégration du site MediGuide et les améliorations d'expérience utilisateur (badges de sources, auto-rename des sessions, section « Mes rendez-vous »).

D. Structure du dépôt

```
.
├─ backend/
│  └─ app/
│     │ └─ database.py      # SQLAlchemy engine + session
│     │ └─ models/models.py # User, Session, Message, Doctor, Appointment
│     │ └─ routes/
│     │    │ └─ auth.py      # /auth/register, /auth/login
│     │    │ └─ chat.py      # /chat/ (RAG + LLM)
│     │    │ └─ sessions.py  # /sessions/* CRUD
│     │    │ └─ hermes.py    # /hermes/recommend, /book, /appointments
│     │    └─ services/
│     │       │ └─ auth.py    # bcrypt + JWT
│     │       │ └─ rag.py     # Pipeline RAG, dispatcher LLM, DuckDuckGo fallback
│     │       └─ hermes.py    # Orchestrateur multi-agents
│  └─ documents/             # 5 PDFs HAS + INCa
│  └─ compare_encoders.py    # Script benchmark 3 encodeurs
│  └─ main.py                # FastAPI entrypoint
│  └─ Dockerfile
│  └─ requirements.txt
├─ frontend/
│  └─ app.py                 # Streamlit (~290 lignes, thème CSS custom)
│  └─ Dockerfile
│  └─ requirements.txt
├─ site/
│  └─ index.html             # Landing MediGuide
│  └─ style.css
│  └─ default.conf          # Conf nginx
│  └─ Dockerfile
├─ deploy/
│  └─ caddy-sae.amanawebagency.com.caddy
├─ docs/
│  └─ rapport.md            # Ce rapport
├─ docker-compose.prod.yml
├─ .env.example
├─ DEPLOY.md                # Procédure de mise en prod détaillée
├─ NOTES_PROJET.md          # Carnet de bord de Yousra
├─ README.md
└─ SKILLS.md                # Référentiel compétences SAE
```

E. Outils utilisés

- **Code** : Python 3.12, FastAPI, Streamlit, LangChain, SQLAlchemy, ChromaDB, sentence-transformers, requests, ddgs, bcrypt, python-jose, google-generativeai.
- **Infra** : Docker, Caddy, Let's Encrypt, rsync, GitHub.
- **Tests** : Playwright (E2E live), curl (smoke tests headers et endpoints).
- **Documentation** : Markdown, pandoc (conversion en .docx et .pdf).

F. Sources documentaires indexées

- Haute Autorité de Santé : <https://www.has-sante.fr>
- Institut National du Cancer : <https://www.e-cancer.fr>

G. Disclaimer

Ce projet est un **exercice pédagogique** dans le cadre de la SAE BUT3 S6. L'assistant MediGuide n'a aucune vocation médicale réelle : ses réponses, bien que basées sur des sources officielles, ne sont pas validées par un professionnel de santé et ne sauraient en aucun cas remplacer une consultation médicale. En cas d'urgence : **15 (SAMU)**.